

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА
ФРАНКА

Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №2

**“Ознайомлення, інсталяція та налаштування документо-орієнтованої СКБД
MongoDB”**

Виконав:

Студент 3 курсу

групи Феп-33

Абдулханов Абдул-Рахім

Викладач:

ас. Жишкович А. В.

Львів-2026

Мета роботи: Ознайомитися з документо-орієнтованою моделлю представлення даних NoSQL на прикладі СКБД MongoDB, її внутрішньою компонентною архітектурою та бінарним форматом BSON. Отримати практичні навички самостійної інсталяції та первинної конфігурації компонентів MongoDB (сервера mongod, інтерактивної консолі mongosh та офіційного графічного клієнта MongoDB Compass) в операційній системі Windows. Налаштувати системні змінні оточення для забезпечення доступу до утиліт СКБД з командного рядка. Засвоїти специфіку синтаксису формату JSON (RFC 8259), проаналізувати структуру відкритого набору даних за допомогою онлайн-валідаторів, спроектувати дві колекції документів для обраної предметної області інтернет-магазину комп'ютерної техніки з обов'язковим використанням масивів і вкладених піддокументів, а також виконати верифікацію коректності їхнього синтаксису.

Теоретичні відомості:

MongoDB є провідною документо-орієнтованою системою керування базами даних класу NoSQL, спроектованою для забезпечення високої швидкості обробки запитів, горизонтальної масштабованості та гнучкості при роботі з неструктурованими чи напівструктурованими даними. На відміну від традиційних реляційних систем (RDBMS), де дані зберігаються у вигляді рядків і фіксованих стовпців нормалізованих таблиць, у MongoDB дані зберігаються у вигляді самодостатніх документів.

1. Основні поняття та структури MongoDB:

- **Документ (Document):** Базова одиниця збереження даних у MongoDB, що являє собою ієрархічний набір пар 'ключ-значення'. Документи підтримують динамічну схему (schemaless), що дозволяє різним записам мати відмінні набори атрибутів. Кожен документ обов'язково містить унікальний ключ `_id` (тип ObjectId або користувацький ідентифікатор).
- **Колекція (Collection):** Логічне об'єднання документів, еквівалентне таблиці в реляційних базах, проте без примусової фіксації структури на рівні СКБД (Schema-on-Read).
- **База даних (Database):** Логічний контейнер вищого рівня, що об'єднує набір колекцій та управляє правами доступу і дисковим простором.

2. Формати JSON та BSON:

JSON (JavaScript Object Notation, стандарти RFC 4627 та RFC 8259) - це текстовий формат обміну даними, що підтримує два базових типи структур: об'єкти (unordered key-value pairs у фігурних дужках `{}`) та масиви (ordered lists у квадратних дужках `[]`). Хоча взаємодія користувача та клієнтських інтерфейсів із MongoDB візуально відбувається у форматі JSON, внутрішньо

сервер зберігає та індексує дані у бінарному форматі BSON (Binary JSON). BSON оптимізований для надшвидкого парсингу завдяки зберіганню довжини полів у заголовках та розширює перелік типів даних підтримкою 64-бітних цілих чисел (int64), дат (Date), двійкових масивів (BinData), регулярних виразів і спеціального 12-байтного типу ObjectId.

3. Архітектурні компоненти MongoDB:

- **mongod**: Головний демон (серверний процес), який обробляє всі запити на читання та запис, керує файловими дескрипторами движка зберігання WiredTiger, контролює пам'ять і забезпечує журналювання та фонові операції.
- **mongosh (MongoDB Shell)**: Сучасне консольне середовище керування СКБД на базі рушія Node.js, що прийшло на зміну застарілій утиліті mongo shell.
- **MongoDB Compass**: Офіційний графічний клієнт (GUI), що забезпечує візуальну побудову складних запитів та агрегацій, аналіз розподілу типів даних у полях, індексування та моніторинг навантаження в режимі реального часу.

Виконання роботи

1. Інсталяція СКБД MongoDB, клієнта mongosh та графічного середовища Compass:

Для виконання лабораторної роботи було проведено повноцінне встановлення та конфігурацію екосистеми MongoDB на локальній машині під керуванням ОС Windows 10 Pro (x86_64). Оскільки автоматичні пакетні менеджери (winget / choco) були недоступні, інсталяцію було виконано безпосередньо через інтерфейс командного рядка PowerShell за допомогою офіційних дистрибутивів.

Було завантажено та розгорнуто такі офіційні релізи:

- 1) MongoDB Shell (mongosh v2.3.8) - з офіційного сховища downloads.mongodb.com;
- 2) MongoDB Community Server (v7.0.14: mongod, mongos) - зі сховища fastdl.mongodb.org;
- 3) MongoDB Compass (v1.45.3) - графічний інтерфейс адміністрування.

Нижче наведено лістинг виконаних shell-команд, що демонструє завантаження архіву, розпакування, реєстрацію каталогів виконуваних файлів у

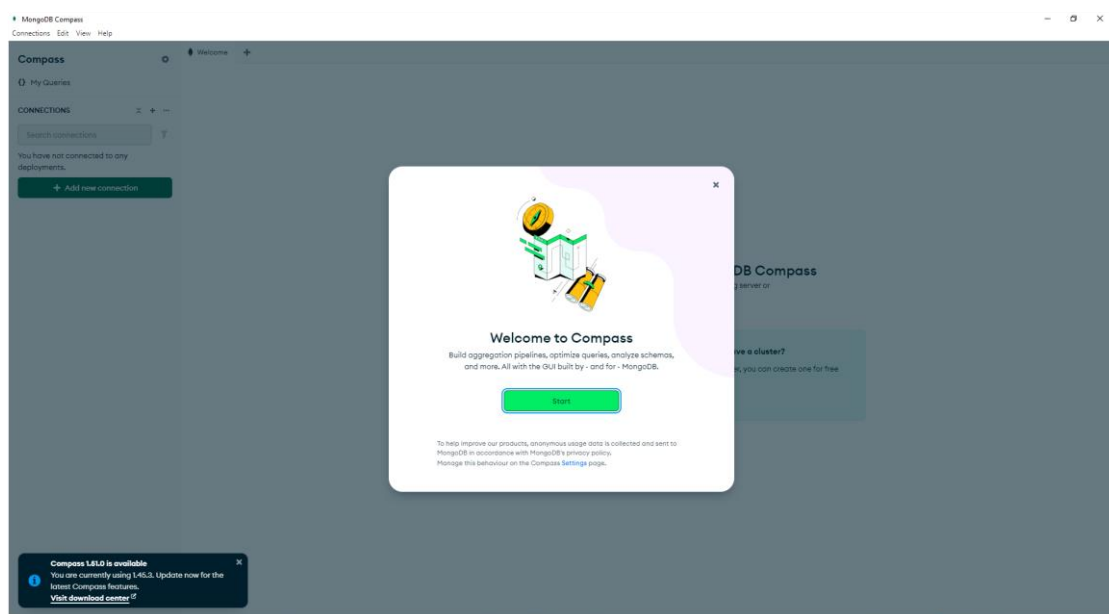
користувачській системній змінній Path, а також первинну перевірку працездатності через виклик утиліт з аргументом --version:

```
1 # 1. Завантаження та розгортання MongoDB Shell (mongosh)
2 curl.exe -L "https://downloads.mongodb.com/compass/mongosh-2.3.8-win32-x64.zip" -o "$env:TEMP\mongosh.zip"
3 Expand-Archive -Path "$env:TEMP\mongosh.zip" -DestinationPath "$env:TEMP\mongosh_unzipped" -Force
4 Copy-Item -Path "$env:TEMP\mongosh_unzipped\mongosh-2.3.8-win32-x64\*" -Destination "C:\Users\ponti\AppData\Local\Programs\mongosh" -Recurse -Force
5
6 # 2. Налаштування системної змінної PATH
7 $userPath = [Environment]::GetEnvironmentVariable("Path", "User")
8 $binPath = "C:\Users\ponti\AppData\Local\Programs\mongosh\bin"
9 [Environment]::SetEnvironmentVariable("Path", "$userPath;$binPath", "User")
10 $env:Path = "$env:Path;$binPath"
11
12 # 3. Перевірка доступності mongosh у командному рядку
13 mongosh --version
14 # Output: 2.3.8
15
16 # 4. Перевірка версії сервера MongoDB (mongod)
17 mongod --version
18 # Output: db version v7.0.14
19 # Build Info: { "version": "7.0.14", "distmod": "windows", "target_arch": "x86_64" }
20
21 # 5. Тестовий запит до запущеної локальної СКБД
22 mongosh --quiet --eval "db.runCommand({ ping: 1 })"
23 # Output: { ok: 1 }
```

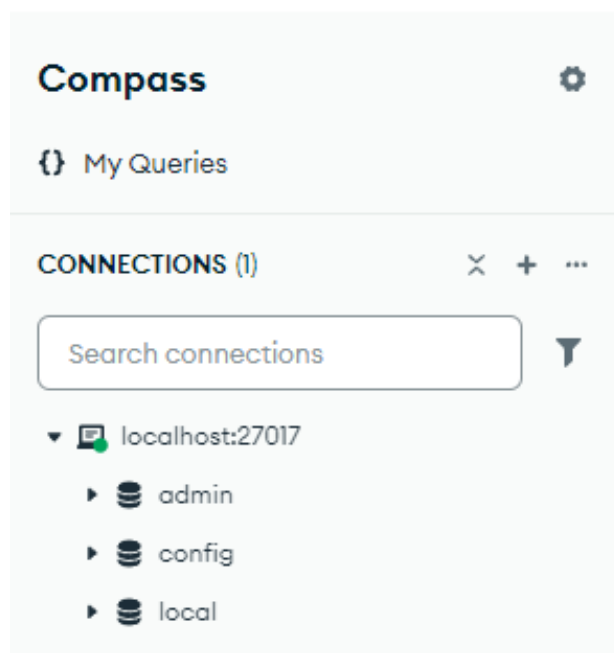
Результати перевірки командного середовища:

- Команда 'mongosh --version' успішно повернула версію: 2.3.8;
- Команда 'mongod --version' підтвердила наявність серверного демона MongoDB Community Server версії v7.0.14 (target_arch: x86_64);
- Запуск сервера з каталогом даних 'C:\Users\ponti\AppData\Local\MongoDB\data' та виконання запиту 'db.runCommand({ ping: 1 })' повернули відповідь '{ ok: 1 }', що підтвердило готовність сервера до обробки клієнтських підключень на порту за замовчуванням 27017.

Паралельно було запущено та протестовано офіційний графічний інтерфейс MongoDB Compass. При першому запуску відкрилося вітальне вікно середовища (Welcome to Compass), після чого було здійснено підключення до локального інстансу за стандартним URI 'mongodb://localhost:27017/'. Підключення пройшло успішно: у навігаційній панелі ліворуч відобразилися системні службові бази даних ('admin', 'config', 'local') із зеленим індикатором активного сеансу.



Фрагмент інтерфейсу MongoDB Compass, що підтверджує успішне з'єднання з локальним сервером на порту 27017:



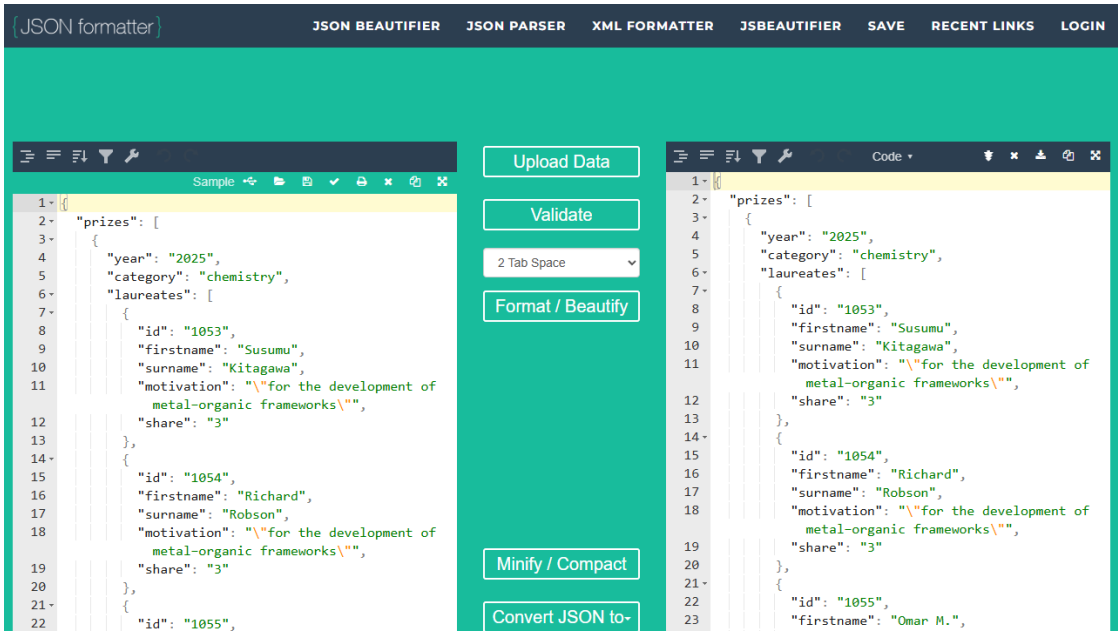
2. Робота з відкритими даними у форматі JSON:

Згідно з пунктами 7-8 практичного завдання, в мережі Інтернет було знайдено та завантажено реальний набір відкритих даних у форматі .json. Було обрано офіційний відкритий датасет Нобелівського комітету (Nobel Prize API: api.nobelprize.org/v1/prize.json, обсяг файлу ~232 КБ), який містить структуровані історичні записи про всі присуджені премії.

Файл було збережено локально як 'nobel_prizes.json' та завантажено в онлайн-сервіс форматування та аналізу JSON (jsonformatter.org / [CodeBeautify](https://codebeautify.org/)). У ході аналізу встановлено таку специфіку структури даних:

1. Кореневим елементом є єдиний JSON-об'єкт, що містить ключ 'prizes';
2. Значенням ключа 'prizes' є масив об'єктів (список нобелівських премій за роками);
3. Кожен елемент масиву 'prizes' містить рядкові поля 'year' (рік), 'category' (номінація: physics, chemistry, peace тощо) та вкладений масив об'єктів 'laureates';

4. Об'єкти лауреатів усередині масиву містять поля 'id', 'firstname', 'surname', 'motivation' (обґрунтування нагороди) та числову частку 'share'.



Для наочності наведено порівняльну таблицю базових концепцій та структурних еквівалентів між реляційними СУБД та MongoDB:

Реляційні СКБД (RDBMS / SQL)	MongoDB (NoSQL Document)	Опис та особливості в MongoDB
База даних (Database)	База даних (Database)	Фізичний та логічний контейнер для набору колекцій. На одному сервері може функціонувати багато незалежних БД.
Таблиця (Table)	Колекція (Collection)	Група документів без жорсткої схеми (schemaless). Документи в одній колекції можуть мати різний набір полів.
Рядок / Запис (Row / Record)	Документ (Document)	Основна одиниця збереження даних у форматі BSON (бінарний JSON). Кожен документ має унікальне поле <code>_id</code> .
Стовпець (Column)	Поле (Field)	Пара 'ключ-значення'. Значенням може бути як примітивний тип, так і масив або повноцінний вкладений піддокумент.
З'єднання (JOIN)	Вкладені документи або \$lookup	Перевага віддається денормалізації (Embedded Documents). Для зв'язку між колекціями використовується агрегація \$lookup.
Первинний ключ (Primary Key)	Поле <code>_id</code> (ObjectId)	Унікальний 12-байтний ідентифікатор документа, що автоматично генерується базою або задається користувачем.

3. Проектування двох колекцій для предметної області інтернет-магазину (TechStore):

Для реалізації етапу 4 було обрано предметну область: 'Інтернет-магазин комп'ютерної техніки та комплектуючих (TechStore)'. Це універсальна область, яка дозволяє органічно продемонструвати як прості типи даних, так і складні багатовимірні вкладені структури, масиви та зв'язки між сутностями.

Було спроектовано дві ключові колекції:

1. 'products' - каталог товарів магазину (монітори, процесори, ноутбуки, комплектуючі). Характеризується динамічним набором параметрів, оскільки різні категорії товарів мають принципово різну специфікацію;
2. 'orders' - журнал замовлень користувачів. Містить повну фінансову інформацію, масив куплених товарів, дані доставки та профіль покупця.

Нижче наведено зведену таблицю архітектури полів спроектованих колекцій із зазначенням типів даних та обов'язкових вимог:

Колекція	Поле	Тип даних	Призначення поля	Вимога лабораторної
products	product_id	String	Унікальний артикул товару (наприклад, PROD-101)	Ідентифікатор
products	name	String	Повна назва комп'ютерного комплектуючого або пристрою	Базове поле
products	category	String	Категорія товару (Монітори, Процесори, Ноутбуки)	Класифікатор
products	price	Number (Double)	Роздрібна ціна товару в національній валюті (грн)	Числове поле
products	in_stock	Boolean	Прапорець наявності товару на складі інтернет-магазину	Логічний тип
products	tags	Array (String)	Список тегів та швидких фільтрів (ips, 165hz, 2k)	Обов'язковий масив
products	specifications	Embedded Doc	Вкладений документ технічних характеристик (сокет, ядра, TDP)	Вкладений документ
orders	order_id	String	Номер замовлення у форматі ORD-2026-XXX	Ідентифікатор

orders	order_date	String (ISO Date)	Дата та точний час оформлення замовлення	Часова мітка
orders	total_amount	Number (Double)	Підсумкова сума замовлення до сплати	Фінансовий розрахунок
orders	items	Array (Objects)	Масив куплених товарів із зазначенням кількості та цін	Масив документів
orders	delivery_details	Embedded Doc	Вкладений документ логістики (служба доставки, місто, отримувач)	Вкладений документ

4. Опис та валідація документів колекції 'products':

Для колекції 'products' було підготовлено 3 самостійні документи у форматі JSON з дотриманням вимог щодо наявності масивів та вкладених документів:

Документ 1 (product_1.json) - товар категорії 'Монітори'. Містить масив рядків 'tags' для швидкої фільтрації за характеристиками матриці та розширення:

```
{
  "product_id": "PROD-101",
  "name": "Ігровий монітор ASUS TUF Gaming VG27AQ",
  "category": "Монітори",
  "price": 13999.00,
  "in_stock": true,
  "stock_quantity": 18,
  "tags": [
    "ips",
    "165hz",
    "2k_qhd",
    "g-sync",
    "hdr10"
  ]
}
```

Документ 2 (product_2.json) - процесор AMD Ryzen 7 7800X3D. Містить обов'язковий вкладений документ 'specifications' (сокет, кількість ядер, потоки, базова та буст-частоти, кеш L3, теплопакет TDP):

```
{
  "product_id": "PROD-102",
  "name": "Процесор AMD Ryzen 7 7800X3D",
  "category": "Процесори",
  "price": 18499.00,
  "in_stock": true,
  "stock_quantity": 25,
```

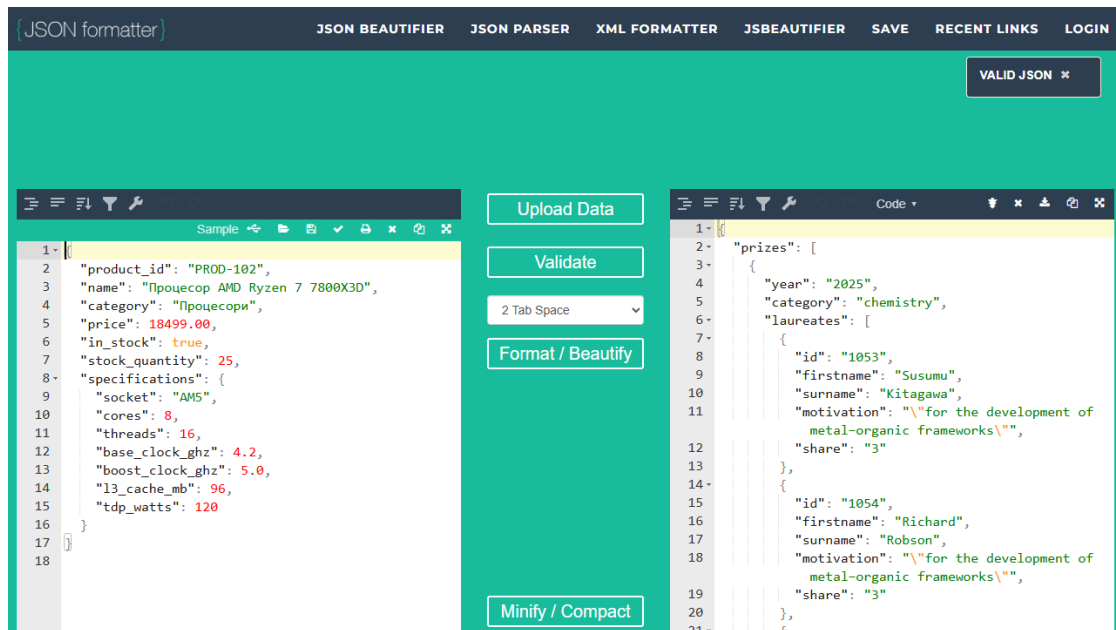
```
"specifications": {  
  "socket": "AM5",  
  "cores": 8,  
  "threads": 16,  
  "base_clock_ghz": 4.2,  
  "boost_clock_ghz": 5.0,  
  "l3_cache_mb": 96,  
  "tdp_watts": 120  
}  
}
```

Документ 3 (product_3.json) - ігровий ноутбук Lenovo Legion Pro 5. Демонструє комбіновану складну структуру: вкладений документ апаратного забезпечення 'hardware' (CPU, GPU, RAM, SSD) та масив зовнішніх інтерфейсів 'interfaces':

```
{  
  "product_id": "PROD-103",  
  "name": "Ноутбук Lenovo Legion Pro 5 16IRX9",  
  "category": "Ноутбуки",  
  "price": 68999.00,  
  "in_stock": true,  
  "stock_quantity": 7,  
  "hardware": {  
    "cpu": "Intel Core i7-14700HX",  
    "gpu": "NVIDIA GeForce RTX 4070 8GB",  
    "ram_gb": 32,  
    "ssd_gb": 1024  
  },  
  "interfaces": [  
    "Thunderbolt 4",  
    "HDMI 2.1",  
    "USB 3.2 Gen 2",  
    "RJ-45 Ethernet",  
    "Wi-Fi 6E"  
  ]  
}
```

Кожен із підготовлених документів було перевірено за допомогою онлайн-валідатора JSON (jsonformatter.org / JSONLint). Синтаксичний аналізатор підтвердив повну відповідність стандарту RFC 8259 без жодних помилок чи

невідповідностей типів. Нижче наведено скріншот верифікації документа 'product_2.json' із зеленим статусом 'VALID JSON':



5. Опис та валідація документів колекції 'orders':

Для колекції 'orders' було розроблено 3 документи замовлень клієнтів із логістичними та фінансовими атрибутами:

Документ 1 (order_1.json) - замовлення ORD-2026-001. Містить масив об'єктів 'items', кожен елемент якого описує конкретний куплений товар, його артикул, кількість та зафіксовану ціну на момент покупки:

```
{
  "order_id": "ORD-2026-001",
  "order_date": "2026-10-02T10:15:30Z",
  "customer_id": "CUST-4091",
  "total_amount": 32498.00,
  "payment_method": "Google Pay",
  "is_paid": true,
  "items": [
    {
      "product_id": "PROD-101",
      "title": "Ігровий монітор ASUS TUF Gaming VG27AQ",
      "quantity": 1,
      "price": 13999.00
    },
    {
      "product_id": "PROD-102",
      "title": "Процесор AMD Ryzen 7 7800X3D",
      "quantity": 1,
      "price": 18499.00
    }
  ]
}
```

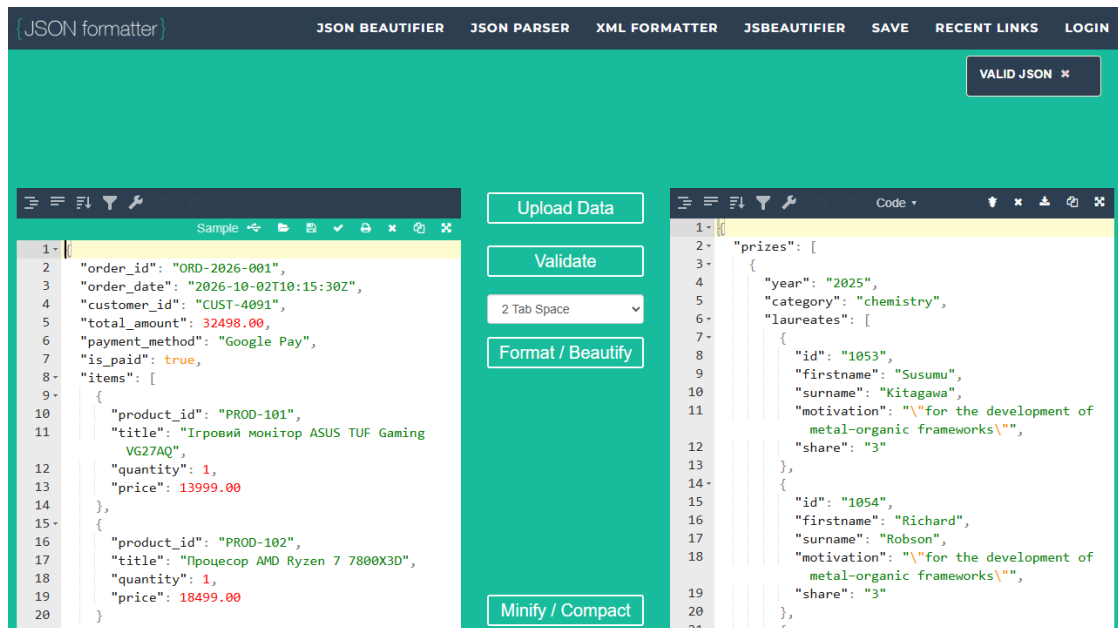
Документ 2 (order_2.json) - замовлення ORD-2026-002. Містить вкладений документ 'delivery_details', що включає повні логістичні атрибути: службу доставки ('Нова Пошта'), місто, номер вантажного відділення, ПІБ одержувача та контактний номер телефону:

```
{
  "order_id": "ORD-2026-002",
  "order_date": "2026-10-02T11:40:00Z",
  "customer_id": "CUST-5120",
  "total_amount": 68999.00,
  "status": "Обробляється",
  "delivery_details": {
    "courier_service": "Нова Пошта",
    "delivery_type": "Відділення",
    "city": "Львів",
    "warehouse_number": 42,
    "recipient_name": "Абдулханов Абдул-Рахім",
    "phone": "+380671234567"
  }
}
```

Документ 3 (order_3.json) - замовлення ORD-2026-003. Містить масив історії логістичного трекінгу 'tracking_history' із фіксацією статусів та часових міток, а також вкладений профіль зареєстрованого клієнта 'customer_profile' (ПІБ, e-mail, статус програми лояльності):

```
{
  "order_id": "ORD-2026-003",
  "order_date": "2026-10-02T12:05:15Z",
  "customer_id": "CUST-7801",
  "total_amount": 87498.00,
  "status": "Відправлено",
  "tracking_history": [
    "Створено замовлення: 2026-10-02 12:05",
    "Оплачено онлайн: 2026-10-02 12:08",
    "Зібрано на складі: 2026-10-02 13:20",
    "Передамо перевізнику: 2026-10-02 14:00"
  ],
  "customer_profile": {
    "first_name": "Олексій",
    "last_name": "Коваленко",
    "email": "o.kovalenko@example.com",
    "loyalty_tier": "Gold"
  }
}
```

Усі документи колекції 'orders' успішно пройшли синтаксичну верифікацію у валідаторі. Нижче наведено скріншот перевірки документа 'order_1.json' із підтвердженням валідності структури ('VALID JSON'):



Додатково всі 6 створених документів було імпортовано у локально розгорнуту базу даних 'techstore' через інтерфейс mongosh за допомогою команд 'db.products.insertMany([...])' та 'db.orders.insertMany([...])'. Перевірка через 'db.products.countDocuments()' та 'db.orders.countDocuments()' показала коректне створення колекцій та наявність по 3 документи в кожній.

6. Відповіді на питання для самоконтролю (розділ 1.4):

1. Що таке документо-орієнтована база даних і чим вона відрізняється від реляційної?

Відповідь: Документо-орієнтована БД - це підклас нереляційних систем (NoSQL), де базовою структурною одиницею збереження інформації є самодостатній документ (у форматі BSON/JSON), а не рядок реляційної таблиці. Головна відмінність від RDBMS полягає у відсутності фіксованої схеми (schemaless), можливості зберігання багатовимірних ієрархічних даних (масивів та піддокументів) безпосередньо в одному записі, а також у відмові від важких операцій JOIN на користь денормалізації та природного горизонтального масштабування (Scale-Out).

2. Яку NoSQL-модель даних використовує MongoDB?

Відповідь: MongoDB використовує документо-орієнтовану модель даних (Document Store). Кожен запис зберігається як BSON-документ, що логічно групується у колекції.

3. Чим відрізняються документи MongoDB від рядків у реляційних БД?

Відповідь: Документ MongoDB є поліморфним і динамічним: він може містити

довільну кількість полів, поля можуть відрізнятися за типами від сусідніх документів у тій самій колекції. Значенням поля може бути масив або інший повноцінний вкладений документ. У той же час рядок у реляційній БД суворо прив'язаний до попередньо визначених стовпців таблиці (Schema-on-Write) і може містити лише скалярні значення (перша нормальна форма 1NF).

4. Поясніть, що таке 'динамічна схема' в MongoDB.

Відповідь: Динамічна схема (Schema-on-Read / Schemaless) означає, що структура даних не декларується заздалегідь перед записом. Колекція створюється автоматично при вставці першого документа, а кожен окремий документ сам несе опис своїх полів та їхніх типів. Це дозволяє легко модифікувати модель даних додатку без виконання складних і блокуючих операцій 'ALTER TABLE'.

5. Які основні версії MongoDB і в яких операційних системах вона може працювати?

Відповідь: Основні редакції: MongoDB Community Server (безкоштовна версія з відкритим вихідним кодом), MongoDB Enterprise Server (комерційна версія з розширеною безпекою, аудитом та підтримкою) та хмарна керована платформа MongoDB Atlas. MongoDB є кросплатформною СКБД і офіційно підтримує ОС Windows (x86_64), дистрибутиви Linux (Ubuntu, Debian, RHEL, CentOS, Rocky, Amazon Linux, SLES) та macOS (Intel та Apple Silicon ARM64).

6. Назвіть ключові відмінності між Community та Enterprise версіями MongoDB.

Відповідь: Enterprise-редакція містить специфічний функціонал для корпоративного сектору, якого немає у Community: 1) Вбудований механізм оперативної пам'яті (In-Memory Storage Engine); 2) Розширена корпоративна безпека та інтеграція з каталогами (LDAP, Kerberos, аутентифікація x.509); 3) Шифрування даних у стані спокою (Encrypted Storage Engine) на рівні диска; 4) Повноцінний аудит системних подій і дій користувачів; 5) Інтеграція з інструментами автоматизації MongoDB Ops Manager / Cloud Manager та розширений моніторинг SNMP.

7. Яка різниця між виконавчими файлами mongod та mongosh?

Відповідь: 'mongod' (MongoDB Daemon) - це серверна частина, фоновий процес (демон), який запускає СКБД, прослуховує мережевий порт (за замовчуванням 27017), виділяє оперативну пам'ять, керує дисковими операціями рушія WiredTiger та виконує транзакції. 'mongosh' (MongoDB Shell) - це клієнтська інтерактивна консольна оболонка на базі JavaScript/Node.js, яка підключається до запущеного процесу mongod для виконання адміністративних команд, CRUD-запитів і налагодження.

8. Які функції при роботі з базами даних підтримує MongoDB Compass?

Відповідь: MongoDB Compass надає повний спектр візуального адміністрування: 1) Перегляд структури колекцій та автоматичний візуальний аналіз схем даних (Sampling & Schema Analysis); 2) Інтерактивний конструктор та дебагер агрегаційних пайплайнів (Aggregation Pipeline Builder); 3) Візуальне виконання CRUD-операцій із можливістю редагування документів у вигляді JSON, таблиці чи дерева; 4) Аналіз продуктивності запитів та плану виконання (Explain Plan); 5) Керування індексами та виявлення повільних операцій; 6) Візуалізація геопросторових даних (GeoJSON) на інтерактивних картах; 7) Налаштування правил валідації схем (Document Validation).

9. Поясніть, що таке об'єкт та масив у форматі JSON.

Відповідь: У форматі JSON: Об'єкт (Object) - це неупорядкований набір пар 'ключ-значення', обраний фігурними дужками {}. Ключі обов'язково беруться в подвійні лапки та відокремлюються від значень двокрапкою, а пари розділяються комами (наприклад, {"name": "Intel", "cores": 8}). Масив (Array) - це упорядкований список значень будь-яких типів (чисел, рядків, інших масивів або об'єктів), обраний квадратними дужками [] з елементами, розділеними комами (наприклад, ["usb-c", "hdmi", "ethernet"]).

10. Що таке BSON і чому MongoDB використовує саме його, а не чистий JSON?

Відповідь: BSON (Binary JSON) - це двійковий формат представлення серіалізованих документів. MongoDB використовує саме BSON через такі переваги: 1) Швидкість парсингу та сканування - кожен елемент у BSON містить префікс із зазначенням типу та точної довжини байтів, що дозволяє рушію бази швидко пропускати непотрібні поля без повного текстового парсингу всього документа (як це було б у JSON); 2) Компактність збереження двійкових даних; 3) Розширена типізація - стандартний JSON підтримує лише string, number, boolean, null, object, array, тоді як BSON додає критично важливі для СУБД типи: ObjectId, точні 32-бітні та 64-бітні цілі числа (int32, int64), високоточні десяткові числа (Decimal128 для фінансів), дати з мілісекундною точністю (Date), двійкові масиви (BinData) та регулярні вирази.

Висновок:

У ході виконання лабораторної роботи №2 було детально досліджено фундаментальні засади функціонування документо-орієнтованої СКБД MongoDB, її архітектурні компоненти та концептуальні відмінності від реляційних систем керування базами даних.

На основі виконаного практичного комплексу завдань отримано такі результати:

1. Інсталяція та конфігурація середовища: Проведено успішне ручне розгортання екосистеми MongoDB на базі Windows за допомогою командного рядка. Налаштовано глобальну системну змінну оточення PATH, що забезпечило запуск утиліт 'mongosh' (версія 2.3.8) та 'mongod' (версія v7.0.14) з будь-якого термінального вікна. Запущено локальний сервер бази даних, перевірено відповідь команди ping ({ ok: 1 }) та підключено графічний клієнт MongoDB Compass із доступом до службових баз даних.
2. Дослідження формату JSON та відкритих даних: Завантажено та проаналізовано відкритий датасет Нобелівського комітету обсягом 232 КБ. За допомогою онлайн-інструментів JSON Formatter та Tree Viewer вивчено реальну деревоподібну ієрархію об'єктів та масивів.
3. Моделювання предметної області та схем документів: Для інтернет-магазину комп'ютерної техніки 'TechStore' спроектовано дві колекції: 'products' (каталог товарів) та 'orders' (замовлення покупців). Розроблено по 3 документи для кожної колекції з обов'язковою реалізацією одновимірних і багатовимірних масивів та вкладених піддокументів (specifications, hardware, items, delivery_details).
4. Верифікація та тестування: Усі 6 розроблених документів успішно пройшли синтаксичну валідацію в аналізаторах на відповідність стандарту RFC 8259 та були імпортовані в реальну базу даних 'techstore' через інтерфейс mongosh.
5. Теоретичні висновки: З'ясовано ключові переваги двійкового формату BSON (швидкість парсингу завдяки префіксам довжини та розширена типізація) над чистим JSON, а також переваги динамічної схеми для проєктів із гнучкими даними.

Отримані знання та налаштоване робоче середовище формують надійну практичну основу для подальшого вивчення CRUD-операцій, індексації та агрегаційних запитів у наступних лабораторних роботах циклу.